

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Сложные методы сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Сортировка слиянием

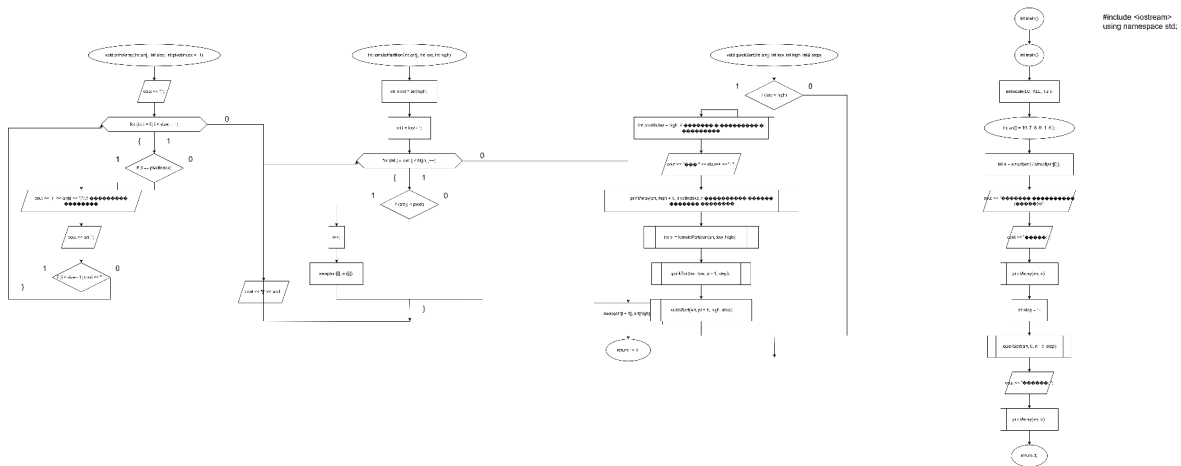
1. Постановка задачи

Отсортировать массив методом слияния

2. Анализ задачи

Единичный массив гарантированно отсортирован - разбиваем исходный массив на единичные массивы и сливаем единичные в 2 эл, потом 2 эл в 4 и тд. - рекурсия!

3. Блок-схема



4. Код

```
#include <iostream>
using namespace std;
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int* L = new int[n1];
```

```
    int* R = new int[n2];
```

```
    for (int i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];
```

```
    for (int j = 0; j < n2; j++)
```

```
        R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k] = L[i];
```

```
            i++;
```

```
        }
```

```
        else {
```

```
            arr[k] = R[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

delete[] L;
delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 38, 27, 43, 3, 9, 82, 10 };
    int size = sizeof(arr) / sizeof(arr[0]);

    cout << "Исходный массив: ";
    printArray(arr, size);

    mergeSort(arr, 0, size - 1);

    cout << "Отсортированный массив: ";
    printArray(arr, size);

    return 0;
}

```

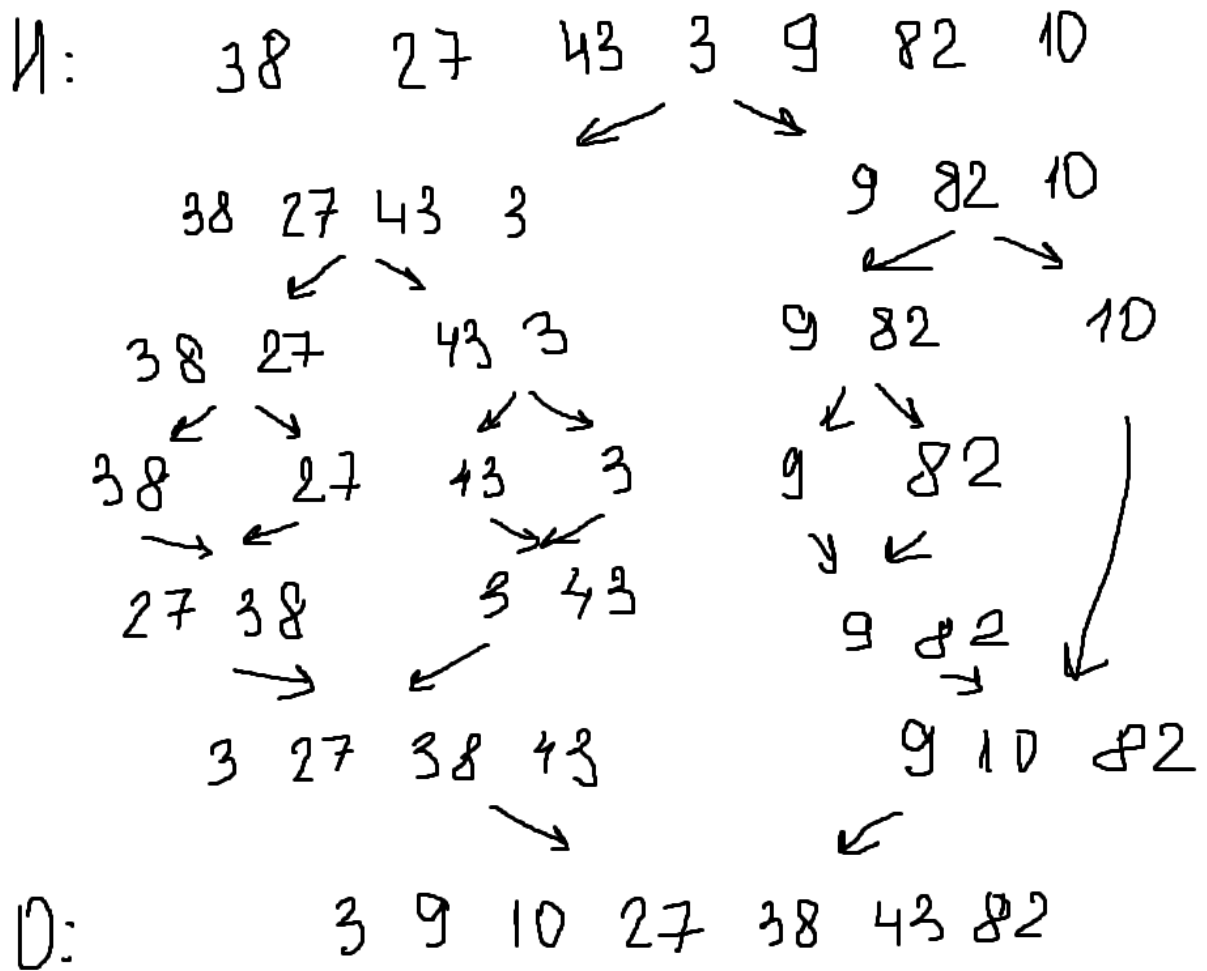
5. Результат выполнения программы

```

Консоль отладки Microsoft Visual Studio
Исходный массив: 38 27 43 3 9 82 10
Отсортированный массив: 3 9 10 27 38 43 82

```

6. Графическое решение



II. Сортировка подсчетом

1. Постановка задачи

Отсортировать массив методом подсчета

2. Анализ задачи

Определение диапазона: Найти минимальный и максимальный элементы в массиве.

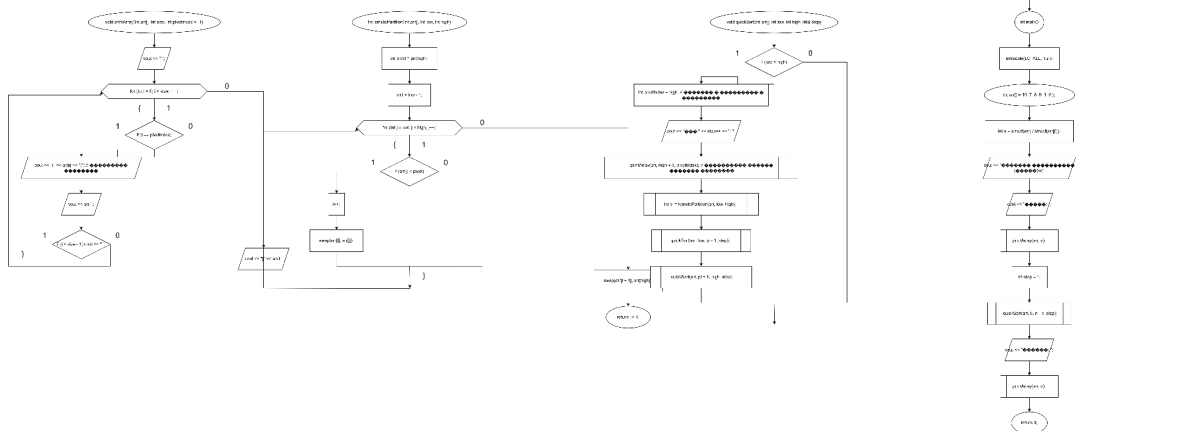
Подсчёт частоты: Создать вспомогательный массив count, где каждый индекс соответствует числу из исходного массива, и заполнить его: записать, сколько раз встречается каждое число.

Накопительные суммы: Преобразовать массив count, записав в каждую ячейку сумму текущего значения и всех предыдущих. Теперь каждое значение в count показывает, на какой позиции (в отсортированном массиве) должно стоять соответствующее число.

Размещение элементов: Создать выходной массив. Пройти по исходному массиву (лучше с конца), для каждого элемента найти его позицию в count, поместить элемент в выходной массив и уменьшить счетчик в count.

Копирование результата: Переписать данные из отсортированного выходного массива обратно в исходный.

3. Блок-схема



4. Код

```
#include <iostream>
#include <vector>
using namespace std;

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void printCountArray(const vector<int>& count, int min_val) {
    cout << " [значение: частота] ";
    for (int i = 0; i < count.size(); i++) {
        cout << (i + min_val) << ":" << count[i] << " ";
    }
}
```

```

    }
    cout << endl;
}

void countingSort(int arr[], int size) {
    if (size <= 1) return;

    cout << "Исходный массив: ";
    printArray(arr, size);

    // ШАГ 1: Находим минимальный и максимальный элементы
    int min_val = arr[0], max_val = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min_val) min_val = arr[i];
        if (arr[i] > max_val) max_val = arr[i];
    }
    int range = max_val - min_val + 1;
    cout << "\nШаг 1 Диапазон: min = " << min_val << ", max = " << max_val
        << " (размер массива count = " << range << ") \n";

    // Создаём массив подсчёта и заполняем нулями
    vector<int> count(range, 0);
    vector<int> output(size);

    // ШАГ 2: Подсчёт частоты каждого элемента
    cout << "\nШаг 2 Подсчёт частот элементов: \n";
    for (int i = 0; i < size; i++) {
        int idx = arr[i] - min_val; // Смещение индекса (чтобы работали и отрицательные числа)
        count[idx]++;
        cout << " arr[" << i << "]=" << arr[i] << " -> увеличиваем count[" << idx << "] \n";
    }
    cout << "Массив count после подсчёта: \n";
    printCountArray(count, min_val);

    // ШАГ 3: Накопительные суммы (префиксные суммы)
    cout << "\nШаг 3 Преобразование в накопительные суммы (позиции в output): \n";
    for (int i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }
    cout << "Массив count после накопления: \n";
    printCountArray(count, min_val);

    // ШАГ 4: Формирование отсортированного массива
    // Проходим С КОНЦА для сохранения устойчивости (стабильности) сортировки
    cout << "\nШаг 4 Размещение элементов в выходной массив (проход с конца): \n";
    for (int i = size - 1; i >= 0; i--) {
        int val = arr[i];
        int idx = val - min_val;
        int pos = count[idx] - 1; // Индекс для размещения (0-based)

        output[pos] = val;
        count[idx]--; // Уменьшаем счётчик, чтобы следующий такой же элемент встал левее

        cout << " arr[" << i << "]=" << val << " -> output[" << pos << "] (count[" << idx << "] стал " << count[idx]
            << ") \n";
    }

    // ШАГ 5: Копирование результата обратно в исходный массив
    cout << "\nШаг 5 Копируем результат в исходный массив: \n";
    for (int i = 0; i < size; i++) {
        arr[i] = output[i];
    }
    cout << "Промежуточный результат: ";
}

```

```

        printArray(arr, size);
    }

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 4, 2, 2, 8, 3, 3, 1 };
    int n = sizeof(arr) / sizeof(arr[0]);

    countingSort(arr, n);

    cout << "\nОтсортированный массив: ";
    printArray(arr, n);

    return 0;
}

```

5. Результат выполнения программы

```

Исходный массив: 4 2 2 8 3 3 1

Шаг 1 Диапазон: min = 1, max = 8 (размер массива count = 8)

Шаг 2 Подсчёт частот элементов:
arr[0]=4 -> увеличиваем count[3]
arr[1]=2 -> увеличиваем count[1]
arr[2]=2 -> увеличиваем count[1]
arr[3]=8 -> увеличиваем count[7]
arr[4]=3 -> увеличиваем count[2]
arr[5]=3 -> увеличиваем count[2]
arr[6]=1 -> увеличиваем count[0]
Массив count после подсчёта:
[значение: частота] 1:1 2:2 3:2 4:1 5:0 6:0 7:0 8:1

Шаг 3 Преобразование в накопительные суммы (позиции в output):
Массив count после накопления:
[значение: частота] 1:1 2:3 3:5 4:6 5:6 6:6 7:6 8:7

Шаг 4 Размещение элементов в выходной массив (проход с конца):
arr[6]=1 -> output[0] (count[0] стал 0)
arr[5]=3 -> output[4] (count[2] стал 4)
arr[4]=3 -> output[3] (count[2] стал 3)
arr[3]=8 -> output[6] (count[7] стал 6)
arr[2]=2 -> output[2] (count[1] стал 2)
arr[1]=2 -> output[1] (count[1] стал 1)
arr[0]=4 -> output[5] (count[3] стал 5)

Шаг 5 Копируем результат в исходный массив:
Промежуточный результат: 1 2 2 3 3 4 8

Отсортированный массив: 1 2 2 3 3 4 8

```

III. Блочная сортировка

1. Постановка задачи

Отсортировать массив методом подсчета

2. Анализ задачи

1. Создание корзин

Выделяется фиксированное количество блоков, каждый покрывает свой диапазон значений (в коде: 10 блоков для 0–9, 10–19, ..., 90–99).

2. Распределение

Проходим по исходному массиву. Каждый элемент кладётся в корзину по формуле:

номер_корзины = значение / 10 (целочисленное деление).

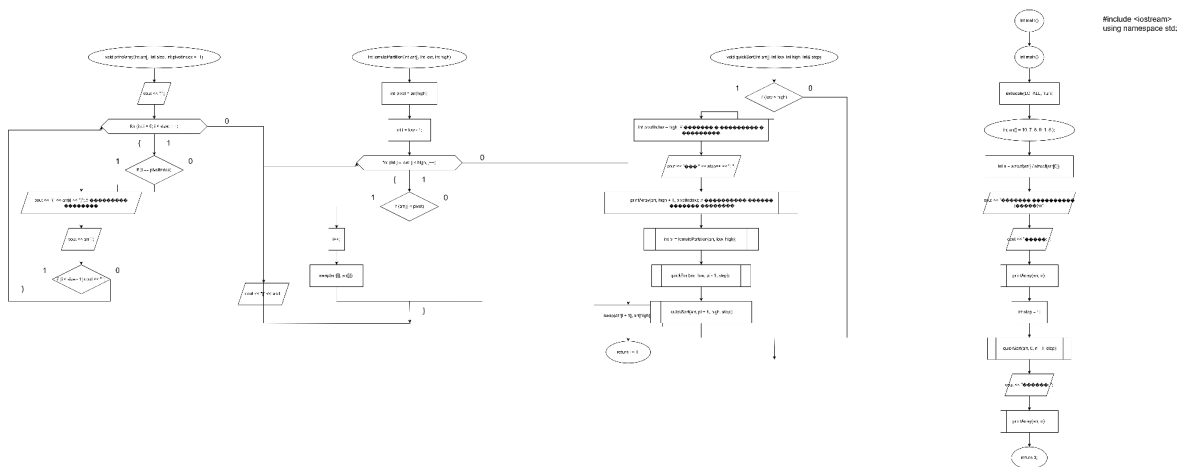
3. Сортировка корзин

Содержимое каждой корзины сортируется отдельно простым методом (в коде — сортировка вставками). Пустые корзины пропускаются.

4. Сборка

Последовательно выгружаем элементы из корзин 0, 1, 2... обратно в исходный массив.

3. Блок-схема



4. Код

```
#include <iostream>
using namespace std;
```

```
const int MAX_N = 20;
const int BUCKETS = 10;
```

```
void printArr(int a[], int n, const char* msg) {
    cout << msg;
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << "\n";
}
```

```
void blockSort(int a[], int n) {
    int blocks[BUCKETS][MAX_N] = { 0 };
    int cnt[BUCKETS] = { 0 };
```



```

printArr(a, n, "Шаг 0 (исходный массив): ");

for (int i = 0; i < n; i++) {
    int b = a[i] / 10;
    if (b >= BUCKETS) b = BUCKETS - 1;
    blocks[b][cnt[b]++] = a[i];
}

cout << "\nШаг 1 (элементы разложены по блокам):\n";
for (int i = 0; i < BUCKETS; i++) {
    cout << "Блок " << i << ": ";
    for (int j = 0; j < cnt[i]; j++) cout << blocks[i][j] << " ";
    cout << "\n";
}

cout << "\nШаг 2 (сортировка блоков по отдельности):\n";
for (int i = 0; i < BUCKETS; i++) {
    for (int j = 1; j < cnt[i]; j++) {
        int key = blocks[i][j];
        int k = j - 1;
        while (k >= 0 && blocks[i][k] > key) {
            blocks[i][k + 1] = blocks[i][k];
            k--;
        }
        blocks[i][k + 1] = key;
    }
    cout << "Блок " << i << " -> ";
    for (int j = 0; j < cnt[i]; j++) cout << blocks[i][j] << " ";
    cout << "\n";
}

int idx = 0;
for (int i = 0; i < BUCKETS; i++)
    for (int j = 0; j < cnt[i]; j++)
        a[idx++] = blocks[i][j];

printArr(a, n, "\nШаг 3 (итоговый отсортированный массив): ");
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 34, 7, 23, 32, 5, 62, 19, 45, 88, 12 };
    int n = sizeof(arr) / sizeof(arr[0]);

    blockSort(arr, n);
    return 0;
}

```

5. Результат выполнения программы

```
Шаг 0 (исходный массив): 34 7 23 32 5 62 19 45 88 12
Шаг 1 (элементы разложены по блокам):
Блок 0: 7 5
Блок 1: 19 12
Блок 2: 23
Блок 3: 34 32
Блок 4: 45
Блок 5:
Блок 6: 62
Блок 7:
Блок 8: 88
Блок 9:
Шаг 2 (сортировка блоков по отдельности):
Блок 0 -> 5 7
Блок 1 -> 12 19
Блок 2 -> 23
Блок 3 -> 32 34
Блок 4 -> 45
Блок 5 ->
Блок 6 -> 62
Блок 7 ->
Блок 8 -> 88
Блок 9 ->
Шаг 3 (итоговый отсортированный массив): 5 7 12 19 23 32 34 45 62 88
```